

Arch-specific SIMD in Go

Junyang Shao and David Chase

2 October 2026

Since Go 1.26, we have supported an experimental `amd64` SIMD API. Now in Go 1.27, we support two more architectures: `arm64` and `wasm`.

The new APIs can be accessed by setting `GOEXPERIMENT=simd` when building a program. They are defined in the `simd/archsimd` package.

`archsimd` is the lower-level infrastructure that `simd` builds on, essentially the intrinsics layer in other languages. For exotic operations that only exist on a specific architecture, users can access them by transitioning from `simd` to `archsimd` (via `ToArch` and `<Types>FromArch`). For details, please refer to the [simd blog](#) post.

For readers with prior knowledge of SIMD intrinsics in other languages, you might find that our `archsimd` API does not directly mirror the underlying hardware instructions:

- We give our APIs sensible names: e.g., instead of `_mm512_slli_epi64`, we just call it `ShiftAllLeft`.
- We leave some architecture details to compiler optimizations rather than exposing them in the API: e.g., instead of `_mm512_maskz_add_ps(m, x, y)`, we optimize `x.Add(y).Masked(m)` into the same instruction.

These design decisions make our APIs smaller and more accessible, and they also happily make the `simd` implementation much easier.

For readers without prior knowledge of SIMD intrinsics, we hope our low-level API is a natural and pleasant journey for you.

Please share your feedback and comments. Anything about usability, performance, etc., is appreciated. The parent issue for this project is [#73787](#).

What is SIMD?

SIMD stands for Single Instruction, Multiple Data. SIMD instructions operate on wide registers (128-bit, 256-bit, or more) containing multiple 8- to 64-bit

elements, performing operations across all elements in parallel. Wider registers and parallel execution provide a significant performance boost to algorithms that can take advantage of them.

Until now, anyone who wanted to use SIMD in Go had to endure the friction of writing assembly language. The new `archsimd` package is here to change that.

Goals¶

The goal is to support as many SIMD instructions across different architectures as possible. Currently, we have `amd64`, `arm64`, and `wasm` available. Our `amd64` support has the largest API surface, covering AVX, AVX2, and many AVX-512 extensions. Our `arm64` support currently covers NEON, with SVE and some SVE2 already on the way. Our `wasm` support is also mostly complete, thanks to its well-defined 128-bit SIMD abstraction.

Some users might be interested in matrix extensions on certain architectures, such as AMX and SME. We haven't fully settled on how to represent matrices effectively in Go, so they are not yet supported, but we might support them in the future.

API Design and Naming¶

Types¶

The architecture-specific SIMD API is imported from `simd/archsimd`. `archsimd` uses distinct struct types, such as `Float32x4`, `Int32x8`, or `Uint8x16`, to signal the shape and element type of each vector, and defines operations as methods on those types. At this moment, `archsimd` supports only fixed-width vector extensions. Scalable vector extensions like `arm64` SVE and RVV will have different struct types to represent vectors. Mask types also come with vector-like shapes, e.g., `Mask32x4`.

Methods¶

Across different vector widths, element types, and architectures, we use the same method name whenever operations are semantically identical:

- Regardless of vector width or architecture, element-wise addition is simply `x.Add(y)`; the type of `x` determines which instruction to emit. Instead of 18 different function names for addition, there is only one method name.
- When instructions on different architectures look similar at a glance but differ in edge-case behavior, we give them distinct names so differences don't silently bite you. For example, 16-byte table lookup on amd64 (`VPSHUFB`) zeros the output byte when the index is negative (`index < 0`) and wraps modulo 16 otherwise, so we call it `Permute0rZero`. On arm64 NEON (`VTBL`) and wasm (`i8x16.swizzle`), any out-of-range index (`index < 0 || index ≥ 16`) zeros the output byte, so we call it `Lookup0rZero`.
- On amd64, many 256-bit and 512-bit instructions operate independently within 128-bit lanes rather than across the entire register. We explicitly suffix those methods with `Grouped` (such as `InterleaveLoGrouped` or `Permute0rZeroGrouped`) to make the 128-bit lane boundary obvious.

Methods require a receiver value, which doesn't work for creating a vector from memory or a scalar. For those, `archsimd` provides package-level functions named with the target vector type:

- Slices (the default): In Go 1.27, loading and storing slices use the shortest names: `archsimd.LoadFloat32x4(s []float32) Float32x4` and `x.Store(s []float32)`. For tail elements that may be shorter than a full vector, `archsimd.LoadFloat32x4Part(s []float32) (Float32x4, int)` zero-fills the remaining lanes and returns the number of elements loaded, paired with `x.StorePart(s []float32) int`.
- Arrays and Broadcasts: Fixed-size array pointers use the `Array` suffix (`archsimd.LoadFloat32x4Array(y *[4]float32)` and `x.StoreArray(y *[4]float32)`), and `archsimd.BroadcastFloat32x4(v float32)` broadcasts a scalar to all lanes.

Type Reinterpretation

Reinterpretations are type casts without any cost. SIMD algorithms frequently reinterpret bits across element types and widths.

In the Go 1.26 experiment, we implemented this with `As<Type>` methods, which suffered from a quadratic explosion of type pairs and couldn't generalize to width-agnostic vectors in `simd` or SVE.

In Go 1.27, we replace them with composable zero-cost conversions:

- `ToBits()` reinterprets a signed integer or float vector as an unsigned integer vector of the same element width (e.g., `Int32x4.ToBits()` → `Uint32x4`), and `BitsToInt32()` / `BitsToFloat32()` converts back.
- `ReshapeToUint<W>s()` on unsigned vectors changes the element width within the same register width. For example, if `x` is an `archsimd.Uint8x16`, then `x.ReshapeToUint32s().BitsToFloat32()` reinterprets the bits of `x` as an `archsimd.Float32x4` at zero runtime cost.

Masks

Different architectures encode vector masks very differently, e.g. `k` mask registers on AVX-512, full vector bitmasks on AVX/AVX2, NEON, and `wasm`, and predicate registers on SVE. To hide this hardware detail, `archsimd` provides opaque `Mask` types (such as `Mask32x4`) matching each vector shape:

- Comparisons like `x.Greater(y)` produce a `Mask`.
- `x.Masked(m)` zeros elements where `m` is false, and `x.IfElse(m, y)` (which replaces `Merge` from Go 1.26) selects elements from `x` where `m` is true and from `y` where `m` is false.
- The compiler peepholes mask operations with the surrounding instructions: on AVX-512, `x.Add(y).Masked(m)` or `x.Add(y).IfElse(m, z)` compiles into a single zero-masked or merge-masked `VPADD` instruction; on AVX2, NEON, or `wasm`, it lowers to the appropriate bitwise `AND` or `blend/bitselect` instruction.

Examples¶

Portable algorithms like inner product can easily be implemented using the portable `simd` package, as shown in the companion [simd blog](#) post. However, we still expose `archsimd` to tap into exotic, architecture-specific instructions that do not fit into a portable intersection API, or instructions that can simply do things faster than what `simd` provides.

On `amd64`, the GFNI extension exposes `GaloisFieldAffineTransform` operations. Putting aside what a Galois Field is, these operations do a simple thing:

```
// Each element of A is interpreted as a matrix of 8 rows
// of vectors of 8 bits.
// Each element of x is interpreted as a vector of 8 bits.
// The b argument is likewise a vector of 8 bits.
// The result is z[i] = (A[i/8] * x[i]) + b.
//
// The matrix * and vector + follow the usual rules for
// linear algebra,
// but based on AND and XOR for scalar * and +
//
// Asm: VGF2P8AFFINEQB, CPU Feature: AVX512GFNI
func (x Uint8x16) GaloisFieldAffineTransform(A Uint64x2, b
uint8) Uint8x16
```

Each 8x8 bit matrix `A` is packed row-by-row into a single `uint64`. Because any linear combination or permutation of the 8 bits within a byte can be written as an 8x8 bit matrix, this single instruction is a Swiss Army knife for byte-level bit manipulation.

For example, reversing the bit order of every byte (`bits.Reverse8`) normally requires nibble lookup tables, masks, shifts, and ORs in SIMD. With

GaloisFieldAffineTransform, we can simply multiply each byte by the 8x8 anti-diagonal identity matrix (0x8040201008040201) to reverse all 8 bits of 64 bytes at a time in a single instruction:

```
/*
  Anti-diagonal 8x8 bit matrix (0x8040201008040201):
  [ 0 0 0 0 0 0 0 1 ] [ x7 ] [ x0 ]
  [ 0 0 0 0 0 0 1 0 ] [ x6 ] [ x1 ]
  [ 0 0 0 0 0 1 0 0 ] [ x5 ] [ x2 ]
  [ 0 0 0 0 1 0 0 0 ] * [ x4 ] = [ x3 ]
  [ 0 0 0 1 0 0 0 0 ] [ x3 ] [ x4 ]
  [ 0 0 1 0 0 0 0 0 ] [ x2 ] [ x5 ]
  [ 0 1 0 0 0 0 0 0 ] [ x1 ] [ x6 ]
  [ 1 0 0 0 0 0 0 0 ] [ x0 ] [ x7 ]
*/
func ReverseBits(dst, src []uint8) {
  if !archsimd.X86.AVX512GFNI() {
    // A slow fallback emulation impl, details omitted.
    slowReverseBits(dst, src)
    return
  }
  if len(src) > len(dst) {
    // Make sure src and dst are the same size
    src = src[:len(dst)]
  }
  revMatrix :=
archsimd.BroadcastUint64x8(0x8040201008040201)
```

```

var v archsimd.Uint8x64
var i int
for i = 0; i < len(src)-v.Len()+1; i += v.Len() {
    v = archsimd.LoadUint8x64(src[i : i+v.Len()])
    v.GaloisFieldAffineTransform(revMatrix,
0).Store(dst[i : i+v.Len()])
}
if i < len(src) {
    v, _ = archsimd.LoadUint8x64Part(src[i:])
    v.GaloisFieldAffineTransform(revMatrix,
0).StorePart(dst[i:])
}
}

```

Another example is matrix transpose. On amd64, the permutation instructions are shaped in a non-portable way, so using `simd` is cumbersome and might incur emulations. An efficient implementation can be written more directly in `archsimd`. The example below transposes an 8-by-8 matrix of 32-bit integers in registers on amd64 using AVX2's 128-bit lane-grouped interleaves (`InterleaveLoGrouped`, `InterleaveHiGrouped`) and cross-lane permutations (`ConcatPermuteScalarsGrouped`, `ConcatPermute128Scalars`):

```

func Transpose8(a0, a1, a2, a3, a4, a5, a6, a7
archsimd.Int32x8) (
    b0, b1, b2, b3, b4, b5, b6, b7 archsimd.Int32x8) {
    if !archsimd.X86.AVX2() {
        // A slow fallback emulation impl, details omitted.
        return slowTranspose8(a0, a1, a2, a3, a4, a5, a6, a7)
    }
}

```

```

    }
    /*
    LOW  HIGH
    a0: abcd efgh
    a1: ijkl mnop
    a2: qrst uvwx
    a3: 0123 4567
    a4: ABCD EFGH
    a5: IJKL MNOP
    a6: QRST UVWX
    a7: 89yz YZ$@
    */
    t0 := a0.InterleaveLoGrouped(a1) // t0 = aibj emfn
    t1 := a0.InterleaveHiGrouped(a1) // t1 = ckdL gohp
    t2 := a2.InterleaveLoGrouped(a3) // t2 = q0r1 u4v5
    t3 := a2.InterleaveHiGrouped(a3) // t3 = s2t3 w6x7
    t4 := a4.InterleaveLoGrouped(a5) // t4 = AIBJ EMFN
    t5 := a4.InterleaveHiGrouped(a5) // t5 = CKDL G0HP
    t6 := a6.InterleaveLoGrouped(a7) // t6 = Q8R9 UYVZ
    t7 := a6.InterleaveHiGrouped(a7) // t7 = SyTz W$X@

    a0 = t0.ConcatPermuteScalarsGrouped(0, 1, 4, 5, t2) //
a0 = aiq0 emu4
    a1 = t0.ConcatPermuteScalarsGrouped(2, 3, 6, 7, t2) //
a1 = bjr1 fnv5

```

```

a2 = t1.ConcatPermuteScalarsGrouped(0, 1, 4, 5, t3) //
a2 = cks2 gow6
a3 = t1.ConcatPermuteScalarsGrouped(2, 3, 6, 7, t3) //
a3 = dlt3 hpx7
a4 = t4.ConcatPermuteScalarsGrouped(0, 1, 4, 5, t6) //
a4 = AIQ8 EMUY
a5 = t4.ConcatPermuteScalarsGrouped(2, 3, 6, 7, t6) //
a5 = BJR9 FNVZ
a6 = t5.ConcatPermuteScalarsGrouped(0, 1, 4, 5, t7) //
a6 = CKSy GOW$
a7 = t5.ConcatPermuteScalarsGrouped(2, 3, 6, 7, t7) //
a7 = DLTz HPX@

b0 = a0.ConcatPermute128Scalars(0, 2, a4) // b0 = aiq0
AIQ8
b1 = a1.ConcatPermute128Scalars(0, 2, a5) // b1 = bjr1
BJR9
b2 = a2.ConcatPermute128Scalars(0, 2, a6) // b2 = cks2
CKSy
b3 = a3.ConcatPermute128Scalars(0, 2, a7) // b3 = dlt3
DLTz
b4 = a0.ConcatPermute128Scalars(1, 3, a4) // b4 = emu4
EMUY
b5 = a1.ConcatPermute128Scalars(1, 3, a5) // b5 = fnv5
FNVZ

```

```

b6 = a2.ConcatPermute128Scalars(1, 3, a6) // b6 = gow6
GOW$
b7 = a3.ConcatPermute128Scalars(1, 3, a7) // b7 = hpx7
HPX@

return
}

```

Good Practices¶

When walking through the examples above, you might have a few questions:

- What is `archsimd.X86.AVX512GFNI()`, and can I omit it?
- Why is the strided loop written as `i = 0; i < len(src)-v.Len()+1; i += v.Len()`, instead of `i = 0; i + v.Len() ≤ len(src); i += v.Len()`?
- Why does `Transpose8` take 8 vector parameters rather than a struct or an `[8]archsimd.Int32x8` array?

At first glance these choices might look like minor stylistic preferences, but each of them is chosen for an important performance or correctness reason:

- CPU feature checks (`archsimd.X86.*`, `archsimd.ARM64.*`)

On `amd64`, we provide runtime feature checks like `archsimd.X86.AVX()`, `AVX2()`, `AVX512()`, and extension checks like `AVX512GFNI()` or `AVX512VNNI()`; on `arm64`, NEON is always available, and we provide checks for optional extensions like `archsimd.ARM64.PMULL()`, `SVE()`, and `SVE2()`. The compiler doesn't know ahead of time whether the machine running your binary supports optional SIMD extensions. If you don't guard your SIMD code with the appropriate feature check, your program may crash with `SIGILL` on hardware that lacks the instruction.

Just as importantly, CPU feature checks act as compiler optimization hints. For example, AVX-512 supports hardware merge-masking on instruction

outputs. How the compiler lowers `x.Add(y).IfElse(m, z)` on a 128-bit or 256-bit vector depends on which CPU features are known to be available. If the code is not guarded by `archsimd.X86.AVX512()`, the compiler must conservatively emit a vector `VPADD` followed by a `VPBLEND` instruction. Inside an `if archsimd.X86.AVX512()` block, however, the compiler knows AVX-512 is available and fuses the sequence into a single merge-masked AVX-512 `VPADD` instruction.

- Bounds check elimination in strided loops

How you write the loop bound determines whether the compiler's prove pass can eliminate slice bounds checks inside the loop. In `i + v.Len() ≤ len(src)`, the addition `i + v.Len()` could theoretically overflow to a negative `int` if `len(src)` were near `math.MaxInt`, which prevents the compiler from proving that `i` is always non-negative and in-bounds. Writing `i < len(src) - v.Len() + 1` avoids that potential overflow, allowing the loop body to compile with zero bounds checks.

- Keeping vectors in registers (avoiding large composite types)

`Transpose8` passes 8 vector parameters rather than a struct of 8 fields or an `[8]archsimd.Int32x8` array because Go's ABI and SSA backend currently place large composite types in memory rather than in registers (a known issue tracked at [#24416](#)). Because SIMD vectors are 16 to 64 bytes each, spilling them to the stack hurts performance much more than spilling scalar values. This applies to both function parameters and local variables, so avoid wrapping SIMD vectors inside large arrays or structs in hot loops. There are ongoing efforts to promote composite memory accesses back into registers, which have not yet landed in Go 1.27.

Trying it out

In Go 1.27, `GOEXPERIMENT=simd` supports `amd64` (AVX, AVX2, and AVX-512), `arm64` (NEON), and `wasm` (WebAssembly 128-bit SIMD) out of the box.

Whether you are on an Intel/AMD machine, an `arm64` server or Apple Silicon Mac (M1/M2/M3/M4), or targeting WebAssembly, you can try `archsimd` natively right away:

```
GOEXPERIMENT=simd go test simd/archsimd/...
```

You can also cross-test `wasm` (or `amd64` via Rosetta emulation on Apple Silicon) by setting `GOARCH`:

```
# Run wasm SIMD tests using a WASI runtime:
```

```
GOOS=wasip1 GOARCH=wasm GOEXPERIMENT=simd go test  
simd/archsimd/...
```

```
# Or run amd64 AVX2 tests on Apple Silicon via Rosetta:
```

```
GOARCH=amd64 GOEXPERIMENT=simd go test simd/archsimd/...
```

We're interested in all sorts of feedback! Early users have already helped us catch bugs (such as [#77582](#)) and identify places where code generation or API ergonomics could be improved. We tried hard to pick clear, consistent names and cover the most useful instructions, but `archsimd` is a massive API surface and there is always room to improve.

Future work

For Go 1.28 and beyond, we are actively working on completing `arm64` SVE and SVE2 support in `archsimd` (introducing width-agnostic scalable vector types like `archsimd.Float32s` backed directly by hardware SVE registers and predicates) and wiring it into `simd`. We also plan to expand `archsimd` to additional architectures such as `riscv64`, `ppc64`, `s390x`, and `loong64`, improve register promotion for composite types, and continue filling in instructions and compiler optimizations based on community feedback.